Packaging for MPEG-DASH

Table of Contents

- Options for MPEG-DASH packaging
- MPEG-DASH profile
- Creating the media files
- Creating the media files with Common Encryption
- Creating the Media Presentation Description file (.mpd)
- Dolby Vision Support
- Playback
- Advanced recipes

Unified Packager can generate fragmented MP4 audio/video and manifests required for MPEG-DASH streaming. We recommend packaging media as CMAF files, see: How to package CMAF, and then creating the media presentation description using the `--package-mpd` option. The following is a list of all options specific to MPEG-DASH packaging.

> **Note**
>
> Package MPEG-DASH compliant media files by either using `--package-mpd` *or* by specifying a CMAF file extension for the output (`.cmfv`, `.cmfa` or `.cmft`), but not both. Previously, we recommended packaging media as `.mp4` files with `--package-mpd` option. This adds a fragment index as `mfra` to the **end** of a file. Going forward, we recommend using CMAF as output. CMAF files have a similar (but distinct) `sidx` index at the **start** of the file.
>
> Packaging CMAF with the `--package-mpd` option (i.e. *mp4split --package-mpd -o video.cmfv video.mp4*) adds **both** `sidx` and `mfra` boxes. It is not recommended to package CMAF files this way (mfra is not used in ISO/IEC 23000-19).
>
> In summary: use `--package-mpd` only to generate the Media Presentation Description (MPD).

Options for MPEG-DASH packaging

--package_mpd

Enable mp4split's MPEG-DASH packaging mode.

--fragment_duration

The duration of each fragment in milliseconds (e.g `8000`) or as a fraction of seconds (e.g. `8000/1000`, `192/24` or `8/1`). Defaults to using the GOP size and if specified manually, it should be a multiple of the GOP size (if it is not, Packager will regard the specified duration as a minimum value).

--segment_list

List the all the segments in a `SegmentList`, instead of using `SegmentBase`.

MPEG-DASH profile

ISO BMFF On Demand

The default MPEG-DASH profile supported by Packager is the ISO Base media file format On Demand profile: `urn:mpeg:dash:profile:isoff-on-demand:2011`.

This is the most basic profile for On-Demand content and thus generates the best compatibility among DASH players and DASH capable devices.

The requirements imposed by this profile are:

- A single segment for each representation (one audio or video track per file).
- Subsegments are aligned across representations (GOP aligned fragments).
- Subsegments must begin with a Stream Access Point (IDR/keyframe).
- The segment is indexed using the Segment Index ('sidx').

In comparison to using a profile like `urn:mpeg:dash:profile:isoff-live:2011` for statically packaged VOD, an important benefit of using the On Demand profile is that it does not segment the content on disk, but rather works with an MPD that specifies segments using byte ranges that point into a contiguous file. This leads to significantly less files and allows for better caching (if a cache preloads some byte range following the requested byte range).

ISO BMFF Main

When the above requirements are not met (for instance when having audio and video in one file), Packager switches to the Main profile: `urn:mpeg:dash:profile:isoff-main:2011`.

Note that not all DASH players support this profile.

ISO BMFF Live

New in version 1.10.5.

Both the On Demand and Main profile described above rely on HTTP byte range requests to work efficiently. For players or CDNs that do not support byte ranges, Packager can generate index-based media segment files alongside an MPD that uses a `SegmentTemplate`, as defined by the DASH Live profile `urn:mpeg:dash:profile:isoff-live:2011`.

When statically packaging DASH according to the Live profile, creating the MPD and packaging the content is done in only one step (instead of packaging the content in a separate step, before creating the MPD). In this one step, the media segments are written to the same directory as the MPD. Therefore, it is recommended to use an empty directory for the output.

To statically package DASH according to the Live profile, enable the `package_mpd` functionality, use your source content as input, specify an `.mpd`-file as output and use the `mpd.profile` option to specify `urn:mpeg:dash:profile:isoff-live:2011`:

```
#!/bin/bash

mkdir out
mp4split --package_mpd \
         -o out/presentation.mpd \
         --fragment_duration=8/1 \
         --mpd.profile="urn:mpeg:dash:profile:isoff-live:2011" \
         tears-of-steel-aac-128k.mp4 \
         tears-of-steel-avc1-400k.mp4
```

Note

When statically packaging content with Packager, the sole purpose of using the `mpd.profile` option is to enable packaging of DASH according to the Live profile. If you specify any other profile it will be ignored.

Creating the media files

First we have to package the audio and video tracks so that it fulfills the requirements listed above:

```bash
#!/bin/bash

mp4split -o tos-aac-128k-dash.cmfa   tears-of-steel-aac-128k.mp4
mp4split -o tos-avc1-1000k-dash.cmfv tears-of-steel-avc1-1000k.mp4
mp4split -o tos-avc1-400k-dash.cmfv  tears-of-steel-avc1-400k.mp4
```

Creating the media files with Common Encryption

Besides preparing content in-the-clear, you can use Common Encryption for the audio and video files. You can use either the DRM system specific command-line options that are available, or make use of CPIX. General info on CPIX can be found in our Content Protection Information eXchange (CPIX) documentation, and how to use if for static packaging is documented in Multi-DRM protected HLS and DASH from a shared CMAF source. Below you will find examples that demonstrate how to use the DRM system specific command-line options for DASH.

> **Note**
>
> In case your input is pre-encrypted, Packager and Origin will pick up on any DRM signaling present in the input and automatically pass it through in the output they generate. This means that for any DRM system for which signaling is present in the input, you do not need to specify DRM configuration options when preparing your stream. However, do note that there are DRM systems for which such signaling can't be present by design, like FairPlay, because signaling for these systems is never stored in the media. This means that to support such DRM systems, you will always need to add the necessary DRM configuration options.

Using 'cenc' Common Encryption

The following example encrypts the files with a single pair of Key ID (KID) and Content Encryption Key (CEK), using the 'cenc' scheme from the Common Encryption specification (i.e., 'CTR' or Counter mode). This encryption scheme is the default.

In this case, the signaling of the content protection is different depending on the DRM systems used. PlayReady signaling requires the License Acquisition URL (LAURL), while Widevine requires the Protection System Specific Header (PSSH) as `drm_specific_data`.

See also Common Encryption (CENC).

```bash
#!/bin/bash

KID=10000000100010001000100000000001
CEK=3a2a1b68dd2bd9b2eeb25e84c4776668
KID_UUID=10000000-1000-1000-1000-100000000001 #UUID representation of KID
CEK_B64="OiobaN0r2bLusl6ExHdmaA==" #Base64 byte array representation of CEK
LAURL="https://test.playready.microsoft.com/service/rightsmanager.asmx?cfg=
(kid:${KID_UUID},contentkey:${CEK_B64},ckt:aesctr)"

PSSH=YOUR_PSSH #Add your Widevine DRM specific PSSH data

mp4split -o tos-aac-128k-dash-cenc.cmfa  \
  --iss.key=${KID}:${CEK} --iss.license_server_url=${LAURL} \
  --widevine.key=${KID}:${CEK} --widevine.drm_specific_data=${PSSH} \
  tears-of-steel-aac-128k.mp4

mp4split -o tos-avc1-400k-dash-cenc.cmfv \
  --iss.key=${KID}:${CEK} --iss.license_server_url=${LAURL} \
  --widevine.key=${KID}:${CEK} --widevine.drm_specific_data=${PSSH} \
  tears-of-steel-avc1-400k.mp4

mp4split -o tos-avc1-1000k-dash-cenc.cmfv \
  --iss.key=${KID}:${CEK} --iss.license_server_url=${LAURL} \
  --widevine.key=${KID}:${CEK} --widevine.drm_specific_data=${PSSH} \
  tears-of-steel-avc1-1000k.mp4
```

Using 'cbcs' Common Encryption

The following example encrypts the files with a single pair of Key ID (KID) and Content Encryption Key (CEK), using the 'cbcs' scheme from the Common Encryption specification (i.e., Cipher Block Chaining mode). This encryption scheme is specified using `--scheme_type=cbcs`.

In this case, the full Protection System Specific Header (PSSH) box must be added for each of the DRM systems that you want to use (except Marlin, which doesn't require such signaling):

```bash
#!/bin/bash

KID=10000000100010001000100000000001
CEK=3a2a1b68dd2bd9b2eeb25e84c4776668

PR_PSSH=<PLAYREADY_PSSH> #Add your PlayReady DRM specific PSSH box
WV_PSSH=<WIDEVINE_PSSH> #Add your Widevine DRM specific PSSH box

mp4split -o tos-aac-128k-dash-cenc.cmfa  \
  --cenc.key=${KID}:${CEK} \
  --scheme_type=cbcs \
  --iss.drm_specific_data=${PR_PSSH} \
  --widevine.drm_specific_data=${WV_PSSH} \
  tears-of-steel-aac-128k.mp4

mp4split -o tos-avc1-400k-dash-cenc.cmfv \
  --cenc.key=${KID}:${CEK} \
  --scheme_type=cbcs \
  --iss.drm_specific_data=${PR_PSSH} \
  --widevine.drm_specific_data=${WV_PSSH} \
  tears-of-steel-avc1-400k.mp4

mp4split -o tos-avc1-1000k-dash-cenc.cmfv \
  --cenc.key=${KID}:${CEK} \
  --scheme_type=cbcs \
  --iss.drm_specific_data=${PR_PSSH} \
  --widevine.drm_specific_data=${WV_PSSH} \
  tears-of-steel-avc1-1000k.mp4
```

Creating the Media Presentation Description file (.mpd)

After the content is properly packaged, step two is creating the Media Presentation Description (MPD) with the packaged media files as input:

```bash
#!/bin/bash

mp4split --package_mpd -o tears-of-steel.mpd \
  tos-aac-128k-dash.cmfa \
  tos-avc1-400k-dash.cmfv \
  tos-avc1-1000k-dash.cmfv
```

If the media is encrypted, content protection signaling is automatically added to the MPD using the protection information stored in the media files. When PlayReady and/or Widevine information is present, these schemes are added to the MPD. Marlin has no explicit signaling in the media files, and is always added to the MPD:

```
#!/bin/bash

mp4split --package_mpd -o tears-of-steel-cenc.mpd \
  tos-aac-128k-dash-cenc.cmfa \
  tos-avc1-400k-dash-cenc.cmfv \
  tos-avc1-1000k-dash-cenc.cmfv
```

Please download the mpeg-dash-cenc.sh sample script that creates the various server manifest as discussed above. The sample content is Tears of Steel.

Adding WebVTT or TTML subtitles

To add subtitles when statically packaging MPEG-DASH, you must first make sure that for each subtitles track you have packaged the WebVTT, SRT or TTML formatted source files in fMP4 (`.ismt` or `.cmft`), as explained in Packaging Subtitles.

When you have packaged your subtitles in fMP4, you simply add each fMP4 that contains a subtitles track that you want to add to your input when creating the MPEG-DASH client manifest (`.mpd`):

```
#!/bin/bash

mp4split -o tos-wvtt-en.cmft \
  tears-of-steel-en.srt --track_language=eng

mp4split --package_mpd -o tears-of-steel.mpd \
  tos-aac-128k-dash.cmfa \
  tos-avc1-400k-dash.cmfv \
  tos-avc1-1000k-dash.cmfv \
  tos-wvtt-en.cmft
```

Adding TTML or WebVTT sidecar subtitles for MPEG-DASH

New in version 1.7.12.

While fragmented MP4 is the format of choice for streaming subtitles, occasionally it may be desirable or necessary to expose raw unsegmented subtitles to the player. In these cases, a WebVTT or TTML *sidecar* file can be added to the MPD.

For instance, the following commands adds German subtitles as WebVTT:

```
#!/bin/bash

mp4split --package-mpd -o movie.mpd \
  [audio/video] \
  subtitles_deu.webvtt --track_language=ger
```

This adds an adaptation set with mime type `text/vtt` (or `application/ttml+xml` for TTML):

```
<AdaptationSet contentType="text" lang="de" mimeType="text/vtt">
  <Representation id="textstream_ger=0" bandwidth="0">
     <BaseURL>subtitles_deu.webvtt</BaseURL>
  </Representation>
</AdaptationSet>
```

Important

When you add sidecar subtitles, they are added as-is. That is, `mp4split` won't addd any metadata from the file. This means that metadata that is of importance for the file should be passed on the command-line (like the track's language, for example).

Dolby Vision Support

New in version 1.7.18.

Packaging MPEG-DASH with Dolby Vision Profile 5, aka Single Layer Non Backwards Compatible (NBC) is supported.

First take the input files containing Dolby Vision NBC to create a fragmented MP4. Then use the fragmented mp4 output to create an MPD:

```
#!/bin/bash

mp4split -o video-with-dolby-vision.cmfv \
  video-with-dolby-vision.mp4

mp4split --package_mpd -o video.mpd \
  video-with-dolby-vision.cmfv \
  example-audio.cmfa
```

Playback

Playback is supported by at least the following sample players:

- dash.js, the DASH IF reference player.
- Shaka player, the Google player.

See Which devices support DASH playback? and Which browsers support DASH playback? for more information.

Javascript / HTML5 based players

The Cross-Origin Resource Sharing (CORS) HTTP headers are necessary when the (javascript) player and content are hosted on different domains.

The are set like the following in the (Apache) virtual host configuration:

```
# Necessary for Media Source Extensions (MSE)
Header always set Access-Control-Allow-Headers "origin, range"
Header always set Access-Control-Allow-Methods "GET, HEAD, OPTIONS"
Header always set Access-Control-Allow-Origin "*"
Header always set Access-Control-Expose-Headers "Server,range"
```

Action Script / Flash based players

For Action Script (Flash) based players the webserver needs to be setup to support Byte-Range requests through a query parameter. This is achieved by rewriting the byte range in the query parameter to the appropriate Range HTTP header in the request. The dash.as player appends '?bytes=' and the dash.js player appends '?range=' to the URL. The two rewrite rules are given in the configuration below.

```
  <IfModule headers_module>
    <IfModule rewrite_module>
      # dash.as
      # ^(.*) if .htaccess in a root; ^/(.*) otherwise
      RewriteEngine On
      RewriteCond %{QUERY_STRING} ^(.*)bytes=(.*)$
      RewriteRule ^(.*) - [L,E=range:%2]
      RequestHeader set Range "bytes=%{range}e" env=range

      # dash.js
      # ^(.*) if .htaccess in a root; ^/(.*) otherwise
      RewriteEngine On
      RewriteCond %{QUERY_STRING} ^(.*)range=(.*)$
      RewriteRule ^(.*) - [L,E=range:%2]
      RequestHeader set Range "bytes=%{range}e" env=range
  </IfModule>
</IfModule>
```

Important

When prepackaging files for MPEG-DASH playout, please make sure that the MP4 files are hosted on a location that doesn't have a running *Unified Origin* handling the MP4 files. Otherwise the progressive MP4 download feature will take over and it'll reformat the prepackaged MP4 files to progressive MP4 instead. This may prevent playout and cause SIDX errors to be displayed by the player.

Advanced recipes

Tuning Common Encryption

--disable_selective_nal_encryption

For NAL structured video samples, only video data in slice NALs should be encrypted with other NAL types kept in the clear.

Specifying this option overrides the (selective) encryption and encrypts all NAL units with a fixed number of bytes in the clear.